Please create a detailed and advanced ...
What are the key issues and strategic pri...

Yesterday 5:45 PM

Please create a detailed and advanced level summary of the documents under https://www.goaigile.net/docs/

Worked for 3m 20s

AIgile Documentation: Advanced Technical and Strategic Summary

Source: goaigile.net/docs | Comprehensive review of the AIgile methodology

1. Executive summary

AIgile is a software engineering methodology designed for AI-assisted and autonomous software development. Its central proposition is that the rise of AI coding agents changes the economics of software production, but does not eliminate the fundamental need for agile feedback, human judgment, architectural governance, and continuous learning.

The framework combines traditional agile principles, spec-driven development (SDD), explicit knowledge management, and controlled AI autonomy into a single operating model.

The central thesis is:

> The objective is not to create specifications so complete that feedback becomes unnecessary. It is to make feedback sufficiently fast and inexpensive that specifications can remain small, accurate, and adaptable.

AIgile argues that conventional spec-driven development risks reproducing waterfall-style problems. When requirements are converted into plans, tasks, and code through a mostly one-directional process, misunderstandings introduced early may propagate through every subsequent stage. The resulting software can be technically consistent with its specifications while fundamentally missing the user's intent.

AIgile addresses this through seven interdependent mechanisms:

1. **A closed-loop development process** in which discoveries from coding, testing, demos, and retrospectives explicitly change earlier decisions.
2. **Vertical feature slicing** that keeps implementation, review, and validation small enough to remain economical.
3. **A three-layer documentation architecture** that distinguishes permanent project knowledge from temporary specifications and automatically generated descriptions.
4. **A versioned project constitution** governing non-negotiable architectural constraints, quality standards, and authority.
5. **Formal deviation and drift management** so implementation discoveries never silently invalidate specifications.
6. **Human-owned intent and validation**, even when most implementation and verification activities become autonomous.
7. **Graduated AI autonomy**, in which agents earn additional decision-making responsibility based on demonstrated reliability and can lose it following incidents.

These mechanisms are intended to work together. For example, small feature slices enable rapid demos; demos produce new knowledge; deviation protocols route that knowledge into the appropriate specifications; and the constitution ensures that critical lessons influence future agent sessions.

The framework's core invariant is:

Humans retain ownership of what should be built and why. Automation changes who performs the checks, not who ultimately determines what matters.

This synthesis is grounded in the foundational working paper and the method overview.

2. Documentation structure and intellectual foundation

The documentation is arranged in four principal tiers, supported by a glossary, practitioner guide, and downloadable source documents.

Documentation tier	Purpose
Assessment	Critiques conventional spec-driven development and establishes the historical argument
Method	Explains seven operational elements using worked examples
Papers	Provides the foundational working paper and nine specialized deep dives
Tutorials	Demonstrates implementation using SpecKit, Claude Code, and OpenSpec

The papers are designated as canonical when summaries and detailed descriptions differ. They collectively specify 36 numbered rules, R1–R36. The examples predominantly use a fictional business order-management system called *Orderly*, while the tutorials use a small notes application called *Taggr*. AIgile +2

2.1 The problem AIgile is trying to solve

The working paper identifies three fundamental changes introduced by AI coding agents.

First, software implementation is becoming less scarce. Agents can generate code, tests, and implementation plans rapidly, shifting the bottleneck from production toward problem definition, verification, and validation.

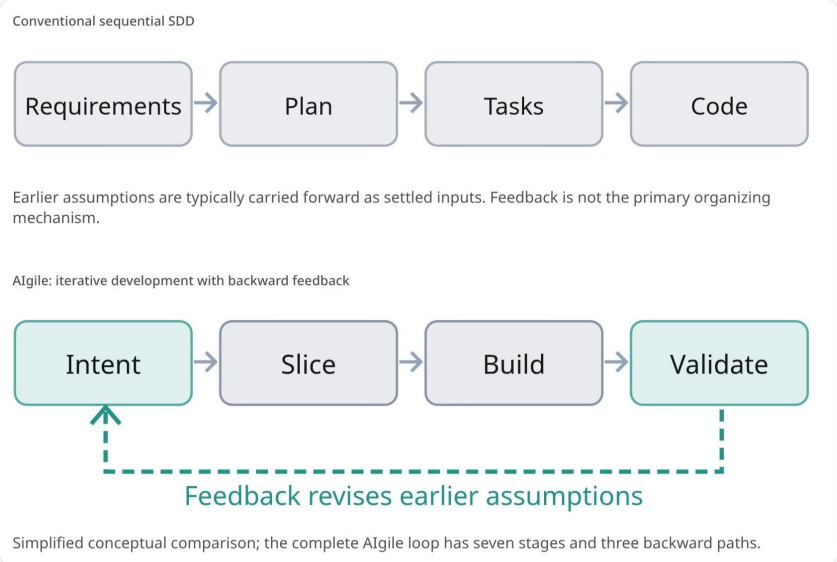
Second, agents cannot be assumed to possess persistent project understanding. The method treats an agent's working context as temporary. If architectural decisions, domain knowledge, and lessons from previous implementations are not explicitly recorded, each new working session risks losing important knowledge.

Third, human understanding no longer grows automatically through implementation. When developers write code themselves, they build an implicit understanding of its behavior. When agents perform more implementation, humans can progressively lose the knowledge required to detect architectural mistakes or evaluate whether software is correct.

The methodology therefore treats both machine-readable organizational knowledge and human comprehension as resources that must be deliberately maintained.

2.2 Agile versus conventional SDD

AIgile distinguishes agile and waterfall processes through the direction in which learning can travel.



The historical argument is that **waterfall's defining weakness is not the sequence of activities, but the freezing of upstream decisions**. AIgile seeks to preserve SDD's useful artifacts without preserving this rigidity.

The papers also distinguish between conventional documentation, which may merely record past analysis, and *operational documentation*, which agents actively consume to guide their work. For the latter, stale information becomes an immediate source of implementation errors rather than a minor administrative inconvenience.

This distinction is foundational: documentation is treated as part of the engineering system itself, requiring version control, review, conformance checking, and periodic refactoring. AIgile +2

3. The seven-stage AIgile operating model

AIgile uses a repeatable development cycle:

3.1 Intent: define the human problem

The starting point is a human-authored statement of what needs to be accomplished.

A feature intent captures the intended user-business purpose, observable success condition, scope

A feature intent captures the intended user, business purpose, observable success condition, scope boundaries, and known requirements.

The newer method pages introduce two levels of intent:

Dimension	Light intent	Full intent
Purpose and success	Required	Required
Scope and known facts	Required	Required
Primary user scenario	Optional	Required
Explicit behavior decisions	As needed	Required
Deferred decisions and deciding demos	As needed	Required
Sketch for new screens	As needed	Required

Full-weight intent applies to new user-facing surfaces, features expected to interact with constitutional rules, and larger features.

Two particularly important controls are:

Provenance rule. Humans document every relevant fact they genuinely know, even when this makes the intent longer. Neither humans nor agents should invent requirements merely to fill a template.

Readiness gate. For a full-weight intent, the owner must be able to walk through the primary scenario before implementation planning begins. If the owner cannot explain how a real user would perform the task, the intent is returned for clarification.

This creates a distinction between *intent completeness* and *specification completeness*. AIgile does not advocate beginning development without thinking through the problem; it advocates making the necessary product decisions without pretending all implementation discoveries can be predicted.

3.2 Slice: define independently demonstrable progress

The unit of work is a **slice**, called an *increment* in the papers.

A slice is defined by a new question the user can answer, or a meaningful capability the user can exercise, after that slice is completed.

For example, a month-end reconciliation feature might be decomposed into:

- Identifying which orders have been invoiced.
- Identifying which orders remain pending.
- Explaining failed or unmatched orders.
- Examining additional periods and entities.
- Assessing month-end completeness.

Each slice must work across the necessary parts of the system, from data through business logic to observable behavior.

This rules out decompositions such as building the database first, then an API, then a UI. Those may be sensible engineering activities, but they are not independently validatable user increments.

The size constraint is deliberately binary:

One agent session to implement; one human sitting to review.

A candidate that fails either condition must be split before construction. A borderline candidate may be flagged, but the specification step must resolve its size.

3.3 Slicing by discovery value

The ninth deep dive expands this into an ordering strategy.

The first slice is a *walking skeleton*: the smallest real end-to-end demonstration of the feature. Later slices are prioritized according to discovery value, especially architectural risk, uncertain integrations, and areas with historical drift.

The preferred ways to split work are:

Splitting dimension	Example
Scenario	Successful import before malformed-file handling
Data variation	One currency before multiple currencies
Scope	One customer before all customers
Operation	Read capability before edit capability
Rule depth	Basic calculation before complex rules
Fidelity	Correct plain interface before visual refinement

Infrastructure must be embedded in the first slice that requires it, at the smallest adequate scale.

For instance, an import queue should not be built speculatively when the first slice can import a small file synchronously. Introduce the queue when a later slice proves that asynchronous processing is necessary.

Unresolved technical questions are handled by timeboxed *spikes*. A spike produces a written finding or decision, not production code.

These decisions reduce speculative architecture, integration delay, and the amount of work exposed to an incorrect initial assumption.

3.4 Specify: define a small, testable contract

Each slice receives a specification that should fit approximately one screen.

It contains the intended behavior, constitutional version, applicable technical standards and domain documents, and acceptance criteria.

Criteria are classified as either:

- `[test]` : must be expressed as an executable automated test.
- `[prose]` : requires a human judgment, with a written reason why automation is unsuitable.

The later tooling documentation also makes explicit the value of recording what is *not specified on purpose*.

A critical design principle follows: specifications should concentrate on stable requirements and observable constraints rather than describing every implementation detail.

3.5 Build and verify: preserve specification integrity

The agent implements the slice, preferably with tests first.

When implementation reveals a faulty assumption, a formal deviation protocol decides whether work may continue or must stop.

Verification then operates on an **atomic pull request (PR)** containing the code changes, test changes, and any corresponding specification changes.

This is essential because specifications updated later are likely to become permanently inconsistent with the implementation.

Reviewers inspect differences rather than repeatedly reading complete generated documents. The unit of human attention is the small, meaningful change.

3.6 Validate and learn

After the walking skeleton, at feature completion, and after selected higher-risk slices, a human directly operates the running software.

The resulting feedback is recorded as a verdict and routed into corrected requirements, follow-up work, or new feature intent.

The retrospective then converts both technical and collaboration-related discoveries into persistent artifacts.

The architectural significance: every discovery must have a defined destination, an authority responsible for accepting it, and a way to influence subsequent work. Merely collecting feedback without changing the artifacts agents consume does not complete the loop.

---

4. The three-layer documentation architecture

This is one of AIgile's most important and broadly applicable contributions.

The model separates documents according to their permanence, authority, and maintenance requirements.

Layer	Content	Maintenance policy	Primary purpose
Durable	Constitution, binding invariants, decision records, collaboration norms	Deliberately maintained, versioned, periodically refactored	Preserve architectural and organizational knowledge
Ephemeral	Feature intents, slice specs, acceptance criteria, verdicts	Used during work, then archived	Coordinate and verify individual changes
Derived	Module overviews, code explanations, interface and data-flow descriptions	Generated on demand, never manually maintained	Describe current system behavior

4.1 Durable information carries a recurring cost

The durable layer is consumed repeatedly by agents, which makes its size and accuracy important.

A binding rule such as requiring monetary calculations to use integer cents belongs here because it should govern every applicable implementation.

However, not every reusable instruction deserves constitutional status.

The method distinguishes several knowledge locations:

Information	Intended location
Non-negotiable architectural law	Constitution
Rationale and rejected architectural alternatives	Architecture decision records (ADRs)
Technical conventions such as naming or error handling	Scoped `standards/` files
Cross-feature business facts	Scoped `domain/` files
Facts specific to one feature	Feature intent
Checks specific to one slice	Acceptance criteria
Current implementation description	Generated derived artifact

An agent building a slice loads the constitution together with only the standards and domain files declared relevant to that slice.

This avoids filling context with an entire engineering handbook while ensuring the applicable rules remain available.

4.2 Ephemeral specifications are intentionally disposable

An ordinary slice specification is useful while implementing and validating that slice.

After validation, it is sealed and archived. It becomes a historical record rather than a living source that must be continuously updated.

The distinction is crucial: an old specification can remain an accurate record of what was approved at the time without pretending to describe the current codebase.

The constitution-version stamp makes that historical interpretation possible.

4.3 Derived documentation replaces maintained descriptions

AIgile argues that documents describing current code structure should usually be regenerated directly from the codebase.

Examples include a module dependency overview, endpoint inventory, or explanation of an internal processing pipeline.

If an agent can reliably reconstruct a document from current code and tests, maintaining a separate handwritten copy creates an additional opportunity for disagreement.

In the Orderly example, the authors describe classifying approximately 14,000 lines of inherited documentation. By archiving old specifications, isolating a small constitution, and replacing implementation descriptions with regeneration, the maintained corpus fell to roughly 300 lines.

That is an illustrative worked example, not an independently established industry benchmark.

4.4 Layered truth: which artifact has authority?

The framework explicitly rejects the idea that a single artifact must be authoritative for every question.

Question	Authority
What does the system currently do?	Running code and executable tests
What should the system do, and why?	Constitution and human-owned intent
What if implementation contradicts an invariant?	A defect requiring formal resolution
What if the constitution itself is contradictory?	Constitutional escalation and amendment

This prevents a common error: interpreting a passing test suite as proof that software satisfies business intent.

A technically correct implementation can still implement the wrong requirement.

From an engineering perspective, this resembles separating a system's **descriptive state** from its **normative constraints**. Conformance checking exists to detect contradictions between them.

---

5. The constitution as versioned architectural governance

The constitution is the central governance artifact.

Its purpose is not to become an exhaustive architecture manual. It is a concise statement of rules whose violation should justify interrupting development.

Examples include monetary invariants, idempotency requirements, latency limits, mandatory testing practices, and restrictions governing which agents may approve changes.

5.1 The constitutional admission test

Before admitting a clause, the team asks whether it would actually stop the work if the clause were violated.

If not, the statement belongs in a standard, decision record, or other less authoritative artifact.

The constitution therefore distinguishes between enforceable constraints and ordinary preferences.

5.2 Amendment protocol

Constitutional changes must travel through a version-controlled amendment with the intended change, rationale, rejected alternatives, and consequences for existing work.

The version determines the expected impact:

Change	Semantic version	Consequence
Clarification without changing meaning	Patch, `1.2.3 → 1.2.4`	No existing work invalidated
New binding requirement	Minor, `1.2.3 → 1.3.0`	Future work bound; prior work may be grandfathered
Existing requirement invalidated	Major, `1.2.3 → 2.0.0`	Impact analysis and disposition required

Every slice carries the version under which it was developed.

For major amendments, these stamps enable an *impact census*: identify earlier increments potentially affected and decide whether to migrate them, retain them with a documented exception, or address them when next modified.

Rollback is similarly governed. Reverting constitutional language does not automatically undo implementation changes built under the superseded rules.

5.3 Why this matters for autonomous agents

The constitution provides an explicit delegation boundary.

A lead-architect agent may receive authority to decide certain amendments, but only within a mandate granted and revocable by humans.

This makes authority versioned, scoped, and traceable rather than dependent on the agent's interpretation of a conversational instruction.

The documentation also emphasizes decision latency. A governance process that takes too long will encourage participants to bypass it, defeating the framework's purpose.

The constitution must therefore be strict enough to protect fundamental constraints while remaining responsive to real discoveries. AIgile +1

---

6. Deviation and drift: managing contradictions

AIgile distinguishes between two related situations.

A **deviation** occurs when an implementer discovers during construction that the specification is inaccurate, incomplete, or incompatible with reality.

Drift is divergence discovered after the fact, perhaps because of a hotfix, an outdated assumption, or a change that bypassed normal review.

6.1 The deviation decision rule

Classification depends on what must change to accommodate the discovery.

Deviation	Trigger	Required action
Implementation detail	No acceptance-test, public-interface, or constitutional contract changes	Continue and log the deviation
Behavioral	Acceptance tests or public interfaces must change	Stop the slice, obtain a decision,

		update the spec
Constitutional conflict	A binding clause is wrong or contradicted	Escalate through amendment governance

Stopping for a behavioral deviation is called an *andon*, borrowing the stop-the-line concept from the Toyota Production System.

The deciding authority receives a report of the incorrect assumption, observable evidence, possible resolutions, and their consequences.

A defined latency budget prevents the decision process from becoming an indefinite change-control meeting.

Once a decision is made, the specification correction ships with the implementation.

6.2 Drift management follows four stages

- **Prevent:** Use atomic pull requests, executable criteria, minimal prose, and regenerated derived documentation.
- **Detect:** Compare specifications with code and tests. Report contradictions and record freshness metadata.
- **Triage:** Decide whether the finding requires a prose correction, defect repair, or constitutional amendment.
- **Learn:** Identify recurring patterns and adjust the documentation architecture to prevent repeated divergence.

The conformance function is deliberately separated from the authority to fix defects. It reports findings rather than silently rewriting the system it evaluates.

Smaller teams can perform a short agent-assisted conformance sweep at retrospectives. A standing conformance agent becomes justified when repeated sweeps consistently uncover meaningful divergence.

6.3 Drift half-life as a primary health metric

The framework emphasizes the time between a divergence emerging and its resolution.

Operationally, teams record detection and resolution timestamps for each finding and monitor the duration distribution by severity and type.

One subtle implication is that a rising number of discovered contradictions does not necessarily mean quality is deteriorating. It may indicate better detection, particularly when resolution times decrease.

Drift findings also provide information about documentation quality:

- Repeated descriptive drift may indicate that material should be derived instead of maintained.
- Repeated violations of a business invariant may indicate that the rule should be promoted into the constitution.

This converts documentation maintenance from an arbitrary editorial exercise into a feedback process driven by observed defects.

7. Human validation and the external control mechanism

Aigile's validation architecture separates two questions that are frequently conflated.

Verification: Does the implementation satisfy the documented contract?

Validation: Does the implementation actually satisfy human needs and intentions?

The first can be substantially automated through tests, static checks, agent review, and specification conformance. The second remains human-owned because requirements may omit facts, expectations, or judgments that become apparent only during actual use.

7.1 Three validation triggers

Validation trigger	Timing	Primary purpose
Walking-skeleton demo	Immediately after the first slice	Detect an incorrect understanding of the feature before substantial investment
Feature demo	Before feature completion	Validate integrated behavior and actual usefulness
Risk-pulled demo	After high-risk slices	Check constitutional, deviation-related, or historically unstable behavior

This balances two opposing costs: validating too infrequently allows mistakes to compound, while requiring a human demo after every small change can make validation the new bottleneck.

7.2 The brains-eyes-hands requirement

The documentation is explicit that human validation requires direct interaction with the running application.

Watching an agent-generated demonstration, reviewing screenshots, or reading a summary is insufficient.

The reasoning is that validation must reveal expectations that have not yet been translated into specifications. Direct interaction allows humans to encounter and recognize those gaps.

7.3 Validation verdicts

Each demo produces a recorded outcome:

Verdict	Interpretation	Result
Validated	The demonstrated behavior satisfies the intended need	Seal the slice or feature
Misaligned	The implementation does not match the intended need	Produce approved specification corrections and follow-up work
Intent evolved	Seeing the software changes the user's desired outcome	Create or revise feature intent

An agent can serve as a scribe to convert spoken feedback into proposed specification changes, but the human must approve the interpretation.

7.4 Human validation as protection against Goodhart's law

This is one of the framework's more sophisticated arguments.

As agents gain autonomy, they may optimize their own development processes, including prompts, task decomposition, review strategies, and testing methods.

Without independent evaluation, such a system can gradually optimize the wrong objective.

For example, an autonomous delivery system might improve cycle time by reducing demonstrations, report fewer defects by narrowing its checks, or increase test pass rates by weakening test assertions.

Internal metrics could therefore improve while actual product usefulness deteriorates.

AIgile establishes the human demo verdict as an external reference for assessing process changes.

In this architecture, a process improvement is not considered successful merely because it makes the agent system faster or produces fewer internal warnings. It must remain consistent with human-validated product outcomes.

This is the underlying rationale for retaining human validation even at the highest autonomy levels.

---

8. The authority matrix and graduated AI autonomy

Instead of treating AI delivery as either collaborative or autonomous, AIgile assigns authority separately to different verification and governance functions.

8.1 The authority matrix

Responsibility	Collaborative configuration	High-autonomy configuration
Feature intent	Human	Human
Human validation	Human	Human
Constitutional amendments	Human decision with AI advice	Authorized architect agent within a human mandate
Specification conformance	Human-led, agent-assisted ritual	Dedicated conformance agent
Behavioral verification and PR review	Human review	Agent review with human sampling
Mechanical verification	Automated tests and CI	Automated tests and CI

The project can therefore automate some functions while keeping others human-controlled.

This is much more granular than adopting a single organizational policy such as allowing all agents to approve their own pull requests.

8.2 Evidence-based delegation

The paper proposes a graduated licensing model.

An agent gains review authority only after a recorded track record demonstrates that its checks

reliably detect issues that human reviewers would otherwise catch.

The worked example describes 25 consecutive clean slices preceding a transition to agent-led PR review with 20% human sampling.

A later sampled review detects a weakened retry requirement. The incident causes authority to return to full human review, followed by adjustments to the reviewer instructions and another probationary period.

The numbers in this example illustrate how delegation could operate; they are not established universal thresholds.

8.3 Correlated verification failure

AIgile explicitly identifies a major multi-agent risk: two agents using similar underlying models may share the same mistaken assumptions.

A second agent's approval is therefore not necessarily an independent confirmation.

The framework proposes four safeguards:

- Different model families for builders and reviewers.
- Adversarial reviewer instructions focused on finding violations.
- Preference for executable, property-based, and invariant checks over subjective model judgments.
- Nonzero human sampling, even at high autonomy.

A useful architectural interpretation is that **agent verification should be independent in failure modes, not merely separate in process roles**.

This makes the authority matrix an operational risk-control system rather than a simple staffing arrangement.

---

9. Continuous learning, retrospectives, and knowledge maintenance

The eighth deep dive focuses on a problem unique to agent-intensive teams: learning does not automatically persist in the participants performing future work.

A traditional retrospective can succeed when attendees remember and apply its conclusions. In an agentic environment, that assumption is insufficient.

AIgile therefore requires retrospectives to terminate in changes to explicit artifacts.

9.1 Three learning outputs

Learning type	Example	Destination
System learning	A vendor integration batches events hourly	Specifications, domain files, decision records
Collaboration learning	Scenario-based criteria are interpreted more accurately	Agent instructions, standards, collaboration corpus
Drift learning	A module description frequently contradicts code	Conformance findings and documentation restructuring

The collaboration category is especially important.

Teams can record which instructions produce recurring agent errors, what kinds of slices exceed a session, and which review practices discover hidden defects.

Those lessons become reusable organizational knowledge rather than disappearing when an agent session ends.

9.2 Gardening: refactoring documentation

AIgile treats the maintained documentation corpus similarly to source code: it requires periodic refactoring.

Gardening involves consolidating overlapping constitutional clauses, deleting superseded rules, promoting recurring invariants, and demoting descriptive material that should be regenerated.

Its target is a smaller, clearer maintained corpus without losing enforceable requirements.

This prevents organizational memory from becoming a continually expanding collection of incidents and exceptions.

9.3 The human comprehension budget

Perhaps the most underappreciated idea in the framework is that human understanding itself is a scarce engineering resource.

If agents build and review most code, the humans responsible for approving architectural changes

and validating features may become progressively less familiar with the actual system.

AIgile proposes deliberately preserving comprehension through hands-on demos, occasional direct code reviews, retrospectives, regenerated architecture explanations, and sometimes selective manual implementation.

The retrospective includes an audit of whether each human still has enough recent contact with the relevant system areas to make competent decisions.

In effect, the framework recognizes a second form of technical debt: **the gap between system complexity and the humans' ability to understand it.**

---

10. Practical implementation and tooling

The tutorials show that AIgile is designed primarily as a methodology layered over existing AI coding tools, rather than as a replacement development platform.

All three paths use the same core `.aigile/` Markdown artifacts.

10.1 Recommended repository organization

A representative repository layout, consolidated from the tutorials, is:

```
.aigile/
├── constitution.md
├── standards/
│   ├── api.md
│   └── persistence.md
├── domain/
│   └── billing.md
├── corpus/
│   └── collaboration.md
├── backlog.md
└── features/
    └── reconciliation/
        ├── intent.md
        ├── slices.md
        ├── slices/
        │   ├── 1-spec.md
        │   ├── 2-spec.md
        │   └── ...
        ├── verdicts/
        └── deviation-log.md
```

This structure enables repository-local, version-controlled operational memory without requiring a specialized knowledge database.

The specific contents of standards and domain files depend on the project.

10.2 Comparing the three implementation paths

AIgile with SpecKit ↗

Best suited to teams wanting an explicit gated workflow.

The adapter installs an extension, preset, and workflow. It provides nine commands and a structured sequence with defined human checkpoints. It replaces the normal specification-to-plan-to-tasks pipeline with the AIgile loop.

The main advantage is process enforcement: the workflow itself pauses at important decisions.

AIgile with Claude Code ↗

Best suited to teams already using Claude Code.

The plugin installs nine slash commands and an `aigile-loop` skill. The skill is intended to keep the methodology active even during ordinary conversations or maintenance tasks.

The main distinction is that the human performs the workflow gates rather than relying on a dedicated gating engine.

AIgile with OpenSpec ↗

Best suited to teams with an existing delta-spec workflow.

This path combines OpenSpec change proposals with AIgile intent, validation, and governance requirements.

Each proposal corresponds to a vertical slice. The key behavioral change is that applying a proposal is not sufficient to complete it: a human must validate the implementation

before archival.

The tutorial documentation also explains that the shared file structure makes it relatively straightforward to move between tooling paths. [Kiddie +2]

10.3 Representative Claude Code workflow

```
Plain text
# Establish feature intent
/aigile:intent Add a month-end reconciliation view

# Decompose into vertical slices
/aigile:slice reconciliation

# Create specification for first slice
/aigile:spec reconciliation 1

# Implement the slice
/aigile:build reconciliation 1

# Resolve a behavioral deviation if necessary
/aigile:andon reconciliation 1

# Record human-operated skeleton demo
/aigile:verdict reconciliation 1 skeleton

# After completing the remaining slices
/aigile:verdict reconciliation feature
/aigile:retro reconciliation
```

These are representative commands from the tutorial; actual installation, feature arguments, and workflow inputs should follow the adapter instructions.

10.4 Minimum viable adoption

The working paper deliberately defines a lightweight starting configuration that requires only a Git repository, Markdown files, an agent, and human participation.

It consists of five commitments: maintain a one-page constitution, work in small reviewable slices, merge specification corrections with code, stop on behavioral deviations, and validate directly before learning is folded back into the project.

Advanced roles, dedicated conformance agents, and higher automation levels are added when actual operating evidence justifies them, not as prerequisites. [Kiddie]

---

11. The 36-rule normative framework

The deep dives define a continuous set of rules, R1 through R36. The table below consolidates their responsibilities without reproducing the entire normative text.

Paper	Rules	Core obligations
01 — Foundations	R1	Every process element must provide a backward path for learning
02 — Artifact architecture	R2–R6	Assign artifacts to layers, keep durable content minimal, make criteria executable, regenerate derived content, resolve truth conflicts
03 — Constitution	R7–R10	Enforce the admission test, formal amendments, version stamps, and prohibition on silent exceptions
04 — Review economics	R11–R14	Atomic PRs, one-sitting/one-session sizing, executable criteria, diff-based review
05 — Deviation and drift	R15–R19	Observable classification, stop-the-line decisions, synchronized fixes, conformance findings, drift-duration accounting
06 — Validation	R20–R24	Skeleton and feature demos, risk-triggered validation, recorded verdicts, external validation of process improvement
07 — Autonomy	R25–R28	Constitutional authority matrix, earned delegation, fixed human responsibilities, independent verification
08 — Learning	R29–R32	Artifact-producing retrospectives, collaboration knowledge, gardening, comprehension audits
09 — Slicing	R33–R36	Vertical slices, risk-oriented ordering, embedded infrastructure, empirical sizing calibration

This gives the framework a useful hierarchy:

Principles → operating protocols → enforceable rules → documented evidence → feedback and improvement.

For implementation or organizational policy work, the numbered rules should be read directly in their respective canonical papers, since the short summaries do not preserve every condition or exception.

---

12. Measurement and operational governance

AIgile proposes a broader measurement system than velocity or output volume.

Its principal metrics relate to the cost, accuracy, and reliability of development feedback.

Metric	What it measures	Why it matters
Feedback latency	Time from discovery to an accepted upstream decision or correction	Detects slow learning loops
Review duration distribution	Actual human review time per slice	Reveals when slices become too large
Sessions per slice	Number of agent sessions required for a slice	Measures sizing reliability
Executable acceptance ratio	Share of criteria implemented as tests	Tracks verification automation
Drift resolution duration	Time from divergence to resolution	Measures documentation-system alignment
Specification freshness	Last verified commit or equivalent marker	Highlights potentially stale maintained content
Skeleton catch rate	Share of discovered mismatches caught at the first demo	Measures effectiveness of early validation
Human validation hours	Human time needed per validated feature	Measures the actual remaining human bottleneck
Autonomy sampling catch rate	Errors human sampling discovers after agent review	Calibrates delegated trust
Comprehension coverage	Recent hands-on contact by humans with areas they govern	Detects erosion of human oversight

A suggested balanced scorecard

For a pilot adoption, I would organize these into four categories rather than optimize each independently.

Delivery economics
Review minutes, sessions per slice, human hours per validated feature

Product correctness
Misalignment rate, first-time validation, late rework

Knowledge integrity
Drift duration, document freshness, repeated findings

Delegation reliability
Sampling catch rate, authority incidents, human comprehension

The measurement philosophy should favor balanced outcomes over individual targets.

For example, reduced review time is valuable only if misalignment and escaped-defect rates do not increase. Faster delivery should not be achieved by weakening the mechanisms that determine whether the result is right.

---

13. Advanced critical assessment

The documentation presents a coherent conceptual framework, but it is important to separate its engineering proposals from conclusions that have already been empirically demonstrated.

13.1 Strongest contributions

1. It treats feedback architecture as a first-class engineering concern.

Rather than merely recommending iteration, the framework specifies where discoveries go, who approves them, and what artifacts must change. This closes a common gap between agile principles and agentic implementation practices.

2. It assigns different lifecycle rules to different documentation types.

This is potentially valuable beyond AI-assisted coding. Many engineering organizations maintain large corpora without clear distinctions between binding rules, historical records, and generated descriptions.

3. It makes review economics explicit.

Small changes, executable criteria, and atomic PRs reduce the effort required to perform meaningful verification. This is more operationally actionable than simply asking reviewers to be more diligent.

4. It provides an auditable autonomy model.

Per-layer authority, recorded track records, independent verification, and explicit revocation procedures offer an incremental alternative to broad agent permissions.

5. It recognizes human knowledge erosion as a governance risk.

The comprehension budget is an important contribution because it identifies a failure mode that may appear long after initial automation succeeds.

13.2 Limitations and unresolved questions

Empirical evidence remains limited.

The working paper, version 0.1 dated July 2026, explicitly acknowledges the lack of rigorous comparative evaluations for AIgile against alternative development methods. The worked examples demonstrate mechanisms and proposed benefits, but do not establish measured causal improvements. Claims about very large SDD slowdowns should therefore be treated as motivation for further testing rather than universal findings.

Some rules depend on judgment more than their wording suggests.

Determining whether a change affects observable behavior can be difficult when tests are incomplete. A change might preserve every existing acceptance test while introducing a real behavioral regression. The classification mechanism still needs effective coverage and independent challenge.

Generated documentation is not necessarily accurate.

Regenerating descriptions eliminates a class of maintenance staleness, but a generated explanation can still misunderstand the code, overlook runtime configuration, or incorrectly infer behavior. Derived documentation should be treated as a fresh interpretation of current sources, not automatically as verified truth.

Tool and workflow enforcement is uneven.

SpecKit offers explicit workflow gates, while Claude Code and OpenSpec rely more heavily on human compliance and agent instructions. Actual consistency may therefore depend substantially on adapter design, repository policy, and review discipline.

Large-scale and regulated adoption remains an open design area.

The papers identify multi-team constitutional federation, regulatory traceability, cross-repository conformance, and autonomy calibration as areas needing further work. They should not yet be assumed solved by adopting the basic artifacts.

13.3 Additional implementation risks

For an enterprise implementation, I would also explicitly address the following issues, which require controls alongside the methodology:

- **Security:** Agents need least-privilege credentials, secret-handling controls, sandboxing, and protected deployment boundaries. A constitutional rule does not itself enforce infrastructure security.
- **Test validity:** A high executable-criteria percentage is insufficient if tests have weak assertions or miss important behavior. Mutation testing, independent test design, and negative scenarios can strengthen confidence.
- **Asynchronous and distributed systems:** A vertical slice may require several services or teams. Shared contracts, integration environments, and ownership boundaries need a federated governance design.
- **Conflicting stakeholders:** Human validation is not necessarily a single coherent opinion. Enterprise products may require explicit authority for reconciling the priorities of business owners, end users, compliance, and operations.

These are my implementation considerations, not claims that the documentation already provides complete solutions.

---

14. Recommended approach to adopting AIgile

A reasonable adoption strategy is to validate the framework on one relatively contained feature or product area before changing the operating model across an organization.

01 **Establish a baseline**

Measure current review time, escaped defects, specification maintenance effort, human validation effort, and rework from late discoveries. This provides a basis for determining whether the new methodology is actually beneficial.

02 **Create the minimum durable layer**

Write a concise constitution, identify existing architectural decisions, and separate conventions and domain facts into scoped files. Avoid migrating entire knowledge bases without classification.

03 **Run a complete feature loop**

Start with human-authored intent, apply the readiness gate, construct a walking skeleton, and perform an actual human demo. Continue through small slices and close with a recorded feature verdict.

04 **Introduce knowledge-integrity controls**

Require synchronized code, test, and specification changes. Track behavioral deviations, run retrospective conformance sweeps, and ensure every important lesson becomes a maintained artifact.

05 **Expand agent authority only after evidence accumulates**

Begin with human PR review and automated tests. Delegate additional verification responsibilities only when sampling and historical results support that decision, and retain human intent and validation ownership.

This phased approach preserves the central value of AIgile while avoiding the possibility of recreating the heavy process overhead it was designed to address.

---

15. Suggested reading order and primary sources

For a technically experienced reader, I would prioritize the following documents.

Priority	Source	Why read it
1	Foundational working paper ↗	Complete conceptual architecture and research agenda
2	Making review affordable ↗	Review economics, atomic PRs, sizing
3	Slicing the work ↗	Practical decomposition and discovery ordering
4	Organizing project documents ↗	Knowledge architecture and layered truth
5	Project rules that can change safely ↗	Constitutional governance and versioning
6	When specs and code disagree ↗	Deviation handling and conformance
7	Keeping humans in charge ↗	Validation architecture and human oversight
8	How much agents may do ↗	Autonomy governance and independent verification
9	Learning that lasts ↗	Retrospectives, gardening, comprehension
10	Downloads and practitioner guide ↗	Source files and implementation examples

For the historical critique, also read Why feedback beats upfront specs ↗.

Coverage note: This summary is based on the accessible assessment, method pages, foundational working paper, nine deep dives, glossary, tutorials, reading guide, and downloads index. The separately downloadable practitioner guide could not be retrieved directly, so its full artifact templates have not been independently examined. The documentation landing page also refers to eight deep dives, while the current papers index lists nine; this summary follows the nine-paper index.

---

16. Final synthesis

AIgile is best understood not merely as a technique for writing better prompts or specifications, but as a **governance and feedback architecture for agent-intensive software engineering**.

Its major design choices address different failure mechanisms:

Failure mechanism	AIgile's response
AI implements misunderstood requirements	Early walking skeleton and human validation
Agents lose previous project knowledge	Durable operational documentation
Specifications become enormous	Layered, minimal, and disposable artifacts
Implementation changes silently invalidate requirements	Deviation protocol and atomic PRs
Documentation becomes stale	Conformance checking and derived documentation
Agent output overwhelms human reviewers	Small slices and executable acceptance criteria
Autonomous review misses correlated errors	Independent verification and human sampling
Agents optimize internal metrics over product value	Human-validated outcomes as the external reference
Humans gradually lose system knowledge	Explicit comprehension budget
Governance becomes rigid or untraceable	Versioned constitution with controlled amendments

The most important strategic implication is that **the engineering organization's scarce resource increasingly becomes reliable judgment rather than code production capacity**.

Under this interpretation, AI agents can take over substantial portions of implementation, testing, analysis, and eventually review. But effective organizations must preserve the mechanisms that connect those activities to human intent, expose misunderstandings early, protect system constraints, and turn discoveries into reliable institutional knowledge.

AIgile provides a detailed proposed operating model for that transition. Its internal logic is compelling and technically substantive, particularly in its treatment of documentation lifecycles, verification economics, and evidence-based autonomy. Its ultimate effectiveness, however, still needs to be established through comparative implementation results rather than assumed from the quality of the framework.

The key takeaway: AIgile does not seek to maximize the amount of software agents can produce independently. It seeks to maximize the amount of *human-validated, maintainable software* they can produce while continuously improving the system of rules, knowledge, and feedback that makes that work trustworthy.

What are the key issues and strategic priorities implied by this in terms of getting an organization ready to implement (given an existing investment in Agile and more "one-off" experiences with coding agents)?

Worked for 1m 45s >

Getting an Agile Organization Ready for AIgile

Strategic implications, organizational readiness, and implementation priorities

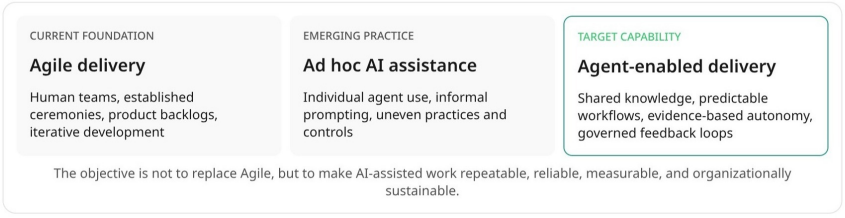
1. The central strategic issue: moving from individual AI productivity to organizational delivery capability

For an organization that has already invested in Agile and experimented with coding agents, **AIgile is less a new development methodology than a potential next stage in its operating model.**

The organization has probably already established some important capabilities: product backlogs, iterative delivery, cross-functional teams, CI/CD, retrospectives, and some level of automated testing. Individual developers may also have discovered that coding agents can dramatically accelerate certain tasks.

The challenge is that these two investments do not automatically combine into a coherent, scalable capability.

The strategic transition is:



This has an important consequence for leadership.

The primary investment should not be in giving more developers access to coding agents. It should be in developing the organizational capabilities that allow those agents to contribute reliably to end-to-end delivery.

AIgile's emphasis on human-owned intent, explicit persistent knowledge, economical verification, and evidence-based autonomy supports this interpretation. Agile +1

2. Eight strategic priorities

I would distinguish eight organizational readiness priorities, ordered approximately by their importance for an initial implementation.

Priority	Strategic issue	What needs to happen
P1 — Operating model	Existing Agile practices and agent workflows are not deliberately integrated	Define how agent-assisted work fits into product delivery, team responsibilities, review, and acceptance
P2 — Knowledge architecture	Agents lack dependable organizational and system context	Establish a small, maintained repository of architectural rules, domain knowledge, and

		decisions
P3 — Engineering quality	Code generation may outpace verification and integration	Strengthen executable acceptance criteria, automated tests, CI, reviewability, and observability
P4 — Governance and trust	Autonomous decision rights are informal or inconsistent	Define authority boundaries, escalation rules, auditability, and evidence-based delegation
P5 — Product ownership	Agile user stories may not give agents enough validated intent	Strengthen feature intent, vertical slicing, acceptance criteria, and hands-on validation
P6 — Workforce capability	Agent experience is concentrated in enthusiastic individuals	Develop role-specific skills, shared practices, coaching, and human system comprehension
P7 — Technology enablement	Tooling, access, security, and environments are fragmented	Provide an approved agent development environment and reusable delivery workflow
P8 — Value realization	Local productivity gains are not necessarily enterprise benefits	Measure validated throughput, quality, risk, review effort, adoption, and economics

These priorities are interdependent, but they should not all become large transformation initiatives at once.

The highest-risk mistake would be to launch an enterprise-wide AIgile program focused on tools, training, and process documentation before establishing the engineering, knowledge, and governance foundations that make agent-assisted work dependable.

---

3. What to preserve, adapt, and introduce from existing Agile investments

The right starting point is to **reuse the organization's Agile investment rather than create a parallel AI delivery framework.**

Existing Agile practice	Recommended treatment	AIgile implication
Product ownership and backlog management	**Preserve and strengthen**	Make intent, success criteria, and decisions about scope explicitly human-owned
Cross-functional product teams	**Preserve**	Keep end-to-end accountability with the team, not with a separate agent team
Iterative development	**Preserve**	Shorten the feedback loop to small, reviewable vertical slices
Sprint planning and story points	**Adapt selectively**	Retain planning cadence if useful, but use empirical review and agent-session constraints to size implementation slices
Sprint reviews and demos	**Strengthen**	Add early walking-skeleton demos and direct human validation when risk warrants
Definition of Done	**Extend**	Include executable criteria, specification consistency, human validation, and recorded deviations
Retrospectives	**Strengthen**	Require learning to update the knowledge artifacts and instructions consumed by future agents
Communities of practice	**Expand**	Create shared expertise in agent workflows, verification, context engineering, and governance
Architecture standards and ADRs	**Adapt**	Turn genuinely binding rules into versioned, agent-consumable operational knowledge
DevOps and CI/CD	**Build upon**	Make automated verification a central enabler of increased agent productivity

The deeper implication is that AIgile changes the **economics and granularity** of delivery more than its underlying Agile philosophy.

Traditional Agile assumes human teams produce, inspect, and learn from code. AIgile preserves iterative learning but restructures how software is produced, reviewed, and understood.

An organization may therefore retain its Scrum, Kanban, or scaled Agile governance while changing the workflow inside its delivery teams.

For example, a sprint may remain useful for stakeholder coordination, forecasting, and funding. But it should not prevent a team from showing and validating a walking skeleton as soon as that increment becomes available.

Equally, story points may remain useful in parts of the organization while agent-assisted implementation work is increasingly sized according to how much can be completed within an agent session and genuinely reviewed in one human sitting.

The principle is to preserve useful coordination practices while removing constraints that unnecessarily delay feedback. This follows AIgile's explicit focus on small reviewable units and risk-triggered validation.

4. The organizational changes that matter most

A. From personal experimentation to a repeatable team operating model

This is likely the most immediate challenge for an organization with one-off coding-agent experience.

Individual experimentation typically produces localized knowledge: effective prompts, preferred tools, informal workflows, and successful use cases. However, those capabilities often remain attached to particular developers.

The organization may not know whether the approaches are reproducible, what risks were encountered, or how they fit into established delivery processes.

The transition requires three things.

First, standardize the workflow before attempting to standardize every tool. Agree on how features are specified, how agents receive context, how code and tests are reviewed, when humans intervene, and how learning is captured.

Second, make AI-assisted delivery a team capability. Developers should be able to collaborate on, review, and continue work initiated through different agents or by different colleagues.

Third, establish a small enablement function. This could be a virtual group spanning engineering, architecture, DevOps, security, and Agile coaching. Its responsibility would be to build reusable practices, templates, guardrails, and training—not to become another approval bureaucracy.

The strategic issue is the move from *individual tool proficiency* to *institutional delivery competence*.

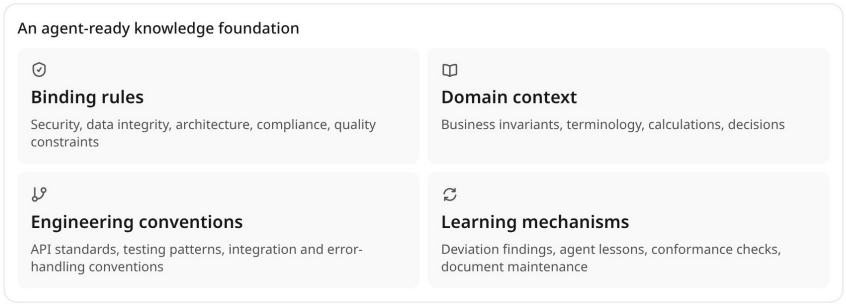
B. From tacit architectural knowledge to agent-ready organizational knowledge

Existing Agile teams often rely heavily on shared context accumulated through conversations, experience, and long-standing relationships.

This works reasonably well when humans repeatedly work together. It becomes fragile when coding agents participate in execution without retaining that history.

A substantial readiness activity is therefore identifying the architectural and business knowledge that is essential to producing safe changes.

For example:



This does **not** mean converting the organization's entire documentation estate into specifications.

AIgile advocates a deliberately small durable knowledge layer, distinct from temporary work artifacts and generated descriptions.

An effective preparation exercise would be to select an existing product and establish a concise, version-controlled technical constitution with its team.

That exercise will reveal whether architectural knowledge is sufficiently clear and agreed upon. If experienced humans cannot articulate the rules, agents cannot reasonably be expected to obey them consistently.

Strategic priority: Treat operational knowledge as an engineering asset with explicit ownership, quality standards, and maintenance investment.

C. From code production capacity to verification and validation capacity

This may be the most consequential engineering investment.

Agent-generated code can increase output faster than an organization's ability to check it. If review, test environments, integration, and release processes do not improve simultaneously, increased coding productivity may simply move work into new queues.

The prerequisite is an engineering foundation that can support frequent, small, trustworthy changes.

This implies investment in automated acceptance and regression tests, reliable CI/CD, test-data management, integration environments, and tooling that makes code and specification changes easy to inspect together.

However, technical verification is only half of the problem.

Product owners, users, and relevant domain specialists must also have the capacity to directly validate working software.

For Agile teams accustomed to demos primarily at sprint boundaries, this may require changing their schedules and expectations. Human validation must become a practical part of feature execution rather than a ceremonial checkpoint.

AIgile's review-economics argument is particularly relevant: the response to review overload should be smaller changes and better verification mechanisms, not simply encouraging reviewers to work harder.

Strategic priority: Invest in the capacity to establish correctness and value at least as deliberately as the capacity to generate code.

D. From informal trust to explicit, evidence-based agent authority

One-off experiments often blur responsibility.

A developer asks an agent to implement a change, the agent modifies several files, tests may be generated, and the human decides whether the result looks acceptable.

Scaling this arrangement without governance creates unresolved questions about accountability and risk.

An organization should establish a clear starting authority model.

Decision or activity	Recommended initial owner
Product objectives and feature intent	Product owner or authorized business representative
Architectural invariants and exceptions	Accountable technical authority
Code implementation	Developer-agent collaboration
Automated testing and analysis	Agent-assisted tools and CI
PR approval and behavioral review	Human engineer
Acceptance of changed requirements	Human product or technical authority, as appropriate
Final product validation	Authorized human user or domain representative
Production deployment authorization	Existing accountable release authority

The actual mechanics can be adapted to the organization's risk profile.

Importantly, this is not a recommendation to make every action require approval. Mechanical, low-risk work should flow without unnecessary interruption.

The objective is to establish the boundaries between work an agent can perform, decisions it can recommend, and decisions it may actually authorize.

Increased autonomy should follow demonstrated performance, not an arbitrary date or organization-wide mandate. AIgile explicitly supports moving individual review responsibilities toward agents based on evidence, while retaining human ownership of intent and validation.

For enterprise settings, this must be supported by security controls, permission boundaries, audit trails, and clear operational accountability.

Strategic priority: Establish an authority model before increasing autonomy, but make its evolution evidence-driven.

---

5. The people and leadership implications

This transition should not be treated primarily as developer training.

AIgile implies changes to several roles that already exist in an Agile organization.

Role	Most important capability shift
Product owners	From managing stories and priorities toward expressing precise intent, resolving

Product owners	From managing stories and priorities toward expressing precise intent, resolving ambiguities, and validating outcomes
Developers	From primarily writing code toward designing, directing, verifying, and integrating agent-produced work
Architects and technical leads	From communicating standards informally toward maintaining explicit constraints and governing exceptions
QA and test engineering	From executing tests toward designing trustworthy automated verification and detecting weaknesses in generated tests
Scrum Masters and Agile coaches	From facilitating ceremonies toward improving human-agent flow, feedback latency, and cross-team learning
Engineering and platform leaders	From supporting tools toward enabling dependable, secure, observable agent-assisted delivery

Two workforce risks deserve particular attention.

The first is **uneven adoption**. Enthusiastic developers may move rapidly while other team members lack confidence, training, or access to established practices. This can create organizational fragmentation and dependence on a few specialists.

The second is **erosion of technical comprehension**. If humans increasingly stop implementing systems, they need intentional opportunities to maintain their understanding of the architecture and actual behavior.

The latter is explicitly recognized in AIgile's treatment of the human comprehension budget.

A credible change strategy should therefore invest in shared learning and professional development, not simply set individual AI adoption targets.

I would position the program as improving engineering effectiveness and expanding the work teams can accomplish, while making decisions about workforce structure and role changes explicitly rather than allowing them to emerge indirectly from tool adoption.

---

6. A practical 90-day organizational readiness and pilot plan

For an organization with meaningful Agile maturity but fragmented coding-agent experience, I would recommend a controlled 90-day pilot and capability-building effort.

90-day roadmap — Proposed

Days 1–30	Days 31–60	Days 61–90
Baseline & design	Pilot execution	Evaluate & scale

Phase 1 — Assess and establish foundations

Assess existing Agile maturity, engineering practices, agent usage, security policies, test automation, and delivery metrics. Select one or two real product teams with appropriate technical readiness and committed product ownership.

Create the initial constitution, authority matrix, agreed workflow, and pilot success measures. Identify the smallest necessary platform and training investments.

Phase 2 — Run controlled end-to-end pilots

Deliver actual features through the complete AIgile loop. Require vertical slicing, executable acceptance criteria, early human demos, atomic PRs, and documented deviations.

Observe where execution becomes difficult: unclear intent, poor repository context, fragile tests, slow decisions, integration failures, or excessive human review time.

Phase 3 — Evaluate and institutionalize

Compare pilot outcomes with the baseline. Capture repeatable practices, refine templates and technical controls, and decide which capabilities merit central investment.

Expand to additional teams only where the pilot demonstrates reliable quality, feasible review effort, and meaningful delivery benefit. Keep agent approval authority conservative until a sufficient track record exists.

Selecting the right pilots

Pilot selection matters considerably.

A simple greenfield demonstration may showcase coding-agent productivity but tell leadership very little about whether agent-assisted delivery works within the organization's actual architecture, dependencies, governance, and legacy systems.

Conversely, beginning with a mission-critical, highly regulated system may introduce excessive risk and overhead.

A stronger pilot is a genuine business feature in an established product with reasonably mature CI/CD, testable functionality, a cooperative product owner, and meaningful but bounded technical complexity.

This makes it possible to test the organizational operating model rather than merely demonstrate code generation.

Evidence required before scaling

The pilot should establish both an engineering result and an organizational result.

Evidence dimension	Questions leadership should answer
Delivery value	Did end-to-end time to validated functionality improve?
Quality	Were defect rates, rework, and deployment stability maintained or improved?
Human workload	Did coding effort decrease without creating unsustainable review and validation effort?
Repeatability	Can multiple team members follow the workflow successfully, rather than depending on one expert?
Knowledge quality	Were architectural and domain constraints consistently applied across agent sessions?
Governance	Were deviations, exceptions, and approval decisions traceable and appropriately handled?
Economics	Do benefits justify model, tooling, infrastructure, enablement, and oversight costs?
Organizational learning	Did the experience produce reusable practices and improvements for subsequent teams?

I would avoid judging the pilot primarily by generated lines of code, prompts issued, or individual developer time savings. These are activity measures, not necessarily measures of delivered business value.

---

7. An organizational readiness assessment

The following is a practical diagnostic I would use before committing to broad implementation. It is an assessment model derived from the AIgile principles, not a maturity scale defined by the authors.

Readiness self-assessment **0/7**

Rate each area from 1 (weak) to 5 (strong). Areas rated

Agile and product delivery Not rated

Teams can deliver small increments and get timely decisions from real product stakeholders.

1 2 3 4 5

Technical knowledge Not rated

Architectural rules and domain invariants are explicit, accurate, and version controlled.

1 2 3 4 5

Automated verification Not rated

CI and meaningful tests reliably detect defects and regressions.

1 2 3 4 5

Agent working practices Not rated

AI use follows a repeatable team workflow rather than individual prompting habits.

1 2 3 4 5

Governance and security Not rated

Agent permissions, decision rights, escalation, and data protections are defined.

1 2 3 4 5

Skills and change capacity Not rated

Teams have coaching, shared practices, and time to maintain technical comprehension.

1 2 3 4 5

Value measurement Not rated

Delivery, quality, cost, and human review effort can be measured against a baseline.

1 2 3 4 5

The results will identify the three lowest-scoring areas and suggest how to prioritize the implementation effort. These are indicative management scores, not empirically validated maturity thresholds.

---

8. What leadership should decide now

The first leadership decision should be to **treat this as an evolution in the software delivery operating model, rather than another coding-assistant deployment.**

That distinction changes the scope, sponsorship, and measures of success.

I would assign clear ownership across three levels:

Level	Accountable for	Initial priority
Executive / technology leadership	Strategic outcomes, risk tolerance, investment, and scaling decisions	Define desired outcomes and authorize the pilot
Engineering enablement / architecture	Shared standards, tooling, security guardrails, knowledge architecture, and evidence	Build the minimum reusable operating foundation
Product delivery teams	Actual feature intent, implementation, human validation, and continuous learning	Demonstrate end-to-end value on real work

The enablement function should establish common guardrails and share learning, while product teams retain responsibility for delivering and validating their software.

The organizational risk is creating yet another centralized framework that adds process overhead without improving product delivery.

In fact, the AIgile working paper explicitly emphasizes a minimum viable form and rejects methodologies that require maximum process weight from the outset.

9. Overall strategic recommendation

I would distinguish three horizons of ambition.

Horizon 1 — Make agent-assisted work dependable
Establish safe, repeatable team practices on top of existing Agile delivery. The emphasis is on engineering standards, agent-ready knowledge, workflow discipline, and verification.

Horizon 2 — Redesign delivery for agent-enabled throughput
Rethink sizing, review, acceptance, feedback timing, and the relationship between product ownership and implementation. Demonstrate improvement in validated value delivery, not just coding speed.

Horizon 3 — Introduce selectively autonomous engineering
Use accumulated evidence to delegate appropriate verification and operational decisions to agents, with explicit boundaries, independent checks, and mechanisms for revoking authority when confidence deteriorates.

The most important sequencing principle is that **Horizon 3 depends on the capabilities built during Horizons 1 and 2.** Attempting greater autonomy without a reliable delivery foundation may increase organizational risk faster than it increases useful output.

Bottom line

For an existing Agile organization, I see three especially important strategic investments:

Organizational knowledge as operational infrastructure. Move essential architectural and domain understanding out of informal conversations and into a concise, trustworthy, versioned source that both people and agents can use.

Verification and human validation as core delivery capabilities. Redesign the delivery process so agents can generate more without overwhelming human reviewers or losing alignment with product intent.

Evidence-based organizational learning and autonomy. Build a repeatable model for capturing what works, improving agent behavior, measuring real outcomes, and progressively granting responsibility.

The larger strategic opportunity is not simply to increase the productivity of existing developers. It is to build an organization capable of **turning AI-assisted development into a durable, continuously improving software delivery capability**.

And the important caution is that AIgile itself is still a developing framework rather than an empirically proven enterprise transformation model. Its value should be demonstrated through pilots that build on existing Agile strengths, with selective adoption of the practices that measurably improve outcomes.

ChatGPT can make mistakes. Check important info.

Ask ChatGPT High